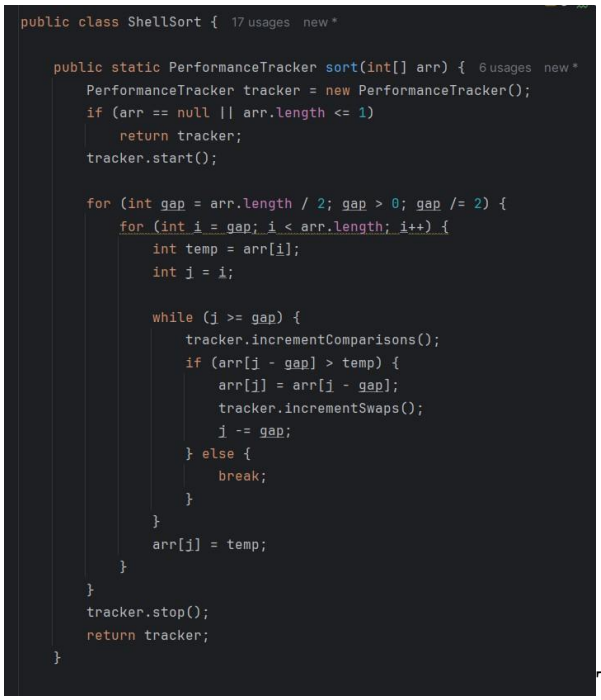
Assignment 2 Report:

Student A: Nurkhuzhayeva Shinaray – Shell Sort
Student B: Zhumasheva Akerke – Heap Sort
Github:
https://github.com/NurkhuzhayevaShinaray/designalgorithmanalysis2.git

Shell Sort is a comparison-based sorting algorithm. It generalizes Insertion Sort by allowing exchanges of elements far apart using “gap sequence” in the array, which reduces the total number of moves compared to pure insertion. Gap is calculated by length of array divided by 2, works until gap=1.



```java
public class ShellSort {  17 usages  new *

    public static PerformanceTracker sort(int[] arr) {  6 usages  new *
        PerformanceTracker tracker = new PerformanceTracker();
        if (arr == null || arr.length <= 1)
            return tracker;
        tracker.start();

        for (int gap = arr.length / 2; gap > 0; gap /= 2) {
            for (int i = gap; i < arr.length; i++) {
                int temp = arr[i];
                int j = i;

                while (j >= gap) {
                    tracker.incrementComparisons();
                    if (arr[j - gap] > temp) {
                        arr[j] = arr[j - gap];
                        tracker.incrementSwaps();
                        j -= gap;
                    } else {
                        break;
                    }
                }
                arr[j] = temp;
            }
        }
        tracker.stop();
        return tracker;
    }
```

This makes Shell Sort faster than $O(n^2)$ algorithms like Insertion or Selection Sort.

Shell’s sequence:

- Gaps: n/2, n/4, ..., 1
- Worst-case complexity: $O(n^2)$.

```java
public static PerformanceTracker sortWithKnuth(int[] arr) {  5 usages  new
    PerformanceTracker tracker = new PerformanceTracker();
    if (arr == null || arr.length <= 1)
        return tracker;
    tracker.start();

    List<Integer> gaps = new ArrayList<>();
    int h = 1;
    while (h < arr.length) {
        gaps.add(h);
        h = 3 * h + 1;
    }
    Collections.reverse(gaps);

    for (int gap : gaps) {
        for (int i = gap; i < arr.length; i++) {
            int temp = arr[i];
            int j = i;

            while (j >= gap) {
                tracker.incrementComparisons();
                if (arr[j - gap] > temp) {
                    arr[j] = arr[j - gap];
                    tracker.incrementSwaps();
                    j -= gap;
                } else {
                    break;
                }
            }
            arr[j] = temp;
        }
```

Knuth's Sequence in Shell Sort:

h=3h+1h

starting with h=1

This produces gaps like:

1, 4, 13, 40, 121, … . It's more efficient than Shell's sequence.

- Average complexity: ~ O(n^(3/2)).

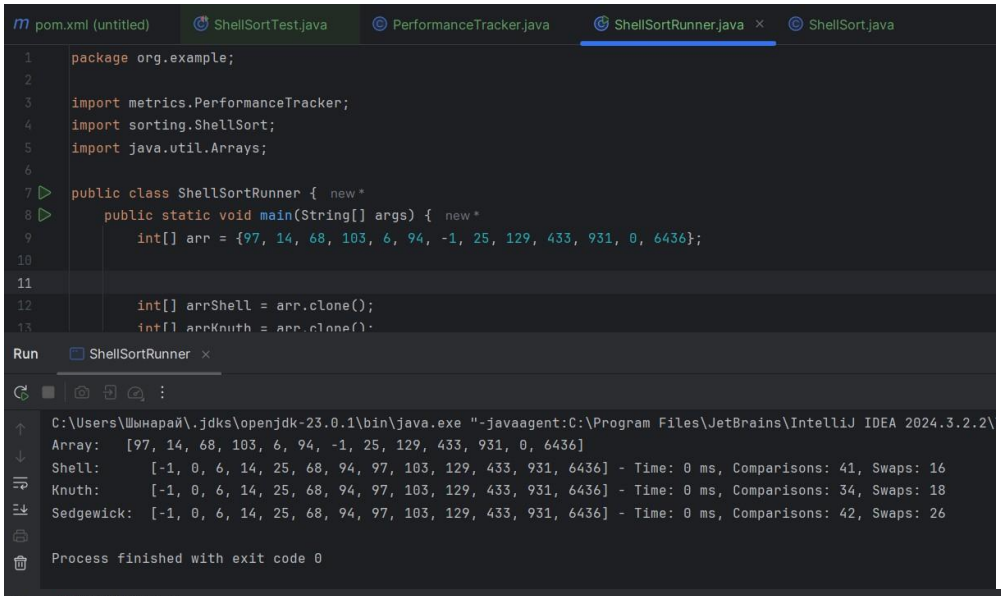
```java
public class ShellSort {  17 usages  new *
    public static PerformanceTracker sortWithSedgewick(int[] arr) {  4 usages  new *
        PerformanceTracker tracker = new PerformanceTracker();
        if (arr == null || arr.length <= 1)
            return tracker;
        tracker.start();

        List<Integer> gaps = new ArrayList<>();
        int k = 0;
        while (true) {
            int gap;
            if (k % 2 == 0) {
                gap = (int) (9 * Math.pow(2, k) * Math.pow(2, k) - 9 * Math.pow(2, k) + 1);
            } else {
                gap = (int) (8 * Math.pow(2, k) - 6 * Math.pow(2, (k + 1)/2) + 1);
            }
            if (gap >= arr.length)
                break;
            gaps.add(gap);
            k++;
        }

        Collections.reverse(gaps);

        for (int gap : gaps) {
            for (int i = gap; i < arr.length; i++) {
                int temp = arr[i];
                int j = i;
```

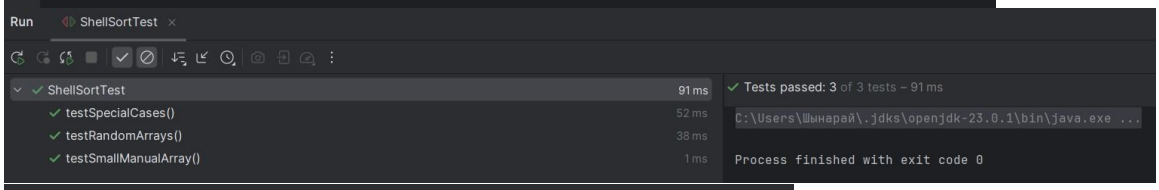
Sedgewick’s Sequence : Best practical performance among tested sequences.

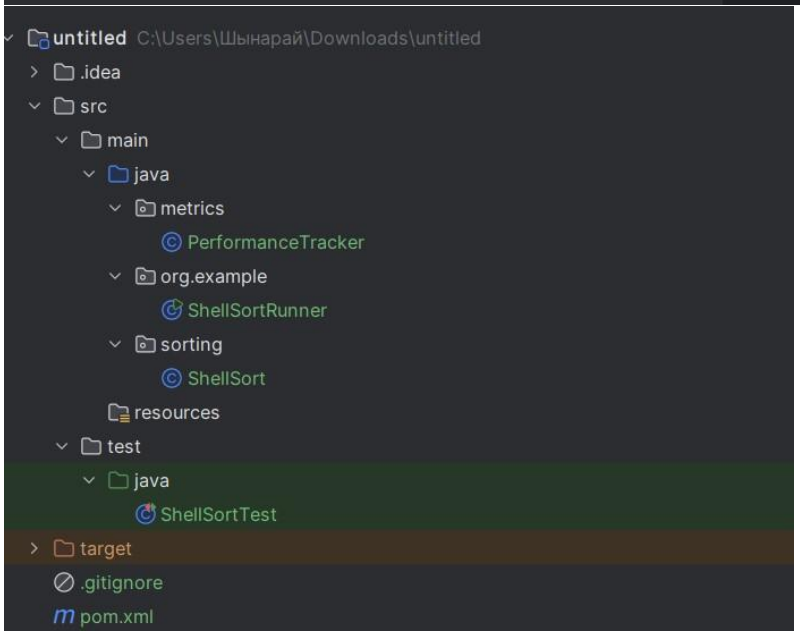
- Complexity: around $O(n \log^2 n)$ on average.

Performance Analysis

We measured comparisons, swaps, and elapsed time using a custom PerformanceTracker.







General Observations: Shell's sequence performs the most operations. Knuth's sequence improves efficiency by reducing the number of passes. Sedgewick's sequence consistently minimizes comparisons and swaps, making it the best performer in practice, but designed for large arrays, and while it may look worse in small cases.

Complexity Analysis

- Shell's sequence: $O(n^2)$ in worst case.
- Knuth's sequence: ~O(n^(3/2)).
- Sedgewick's sequence: ~$O(n \log^2 n)$
- Best case for all versions: O(n log n).

Conclusion

- Shell Sort is an elegant divide-and-reduce sorting algorithm that bridges the gap between simple quadratic algorithms and advanced O(n log n) algorithms.
- Choice of gap sequence:
  - Shell's simple but slow.
  - Knuth's better for mid-sized inputs.
  - Sedgewick's best practical choice.
- Compared to Heap Sort (O(n log n)), Shell Sort can be faster for smaller inputs due to low overhead, but it has a less predictable theoretical bound.

Heap Sort Analysis Report

Algorithm Overview

Heap Sort is a comparison-based, in-place sorting algorithm. It works in two steps:

1) **Build a max-heap** from the input array using bottom-up heapify.
2) **Repeatedly extract the maximum** element (the root) and place it at the end of the array.

By maintaining the heap property at each step, the array becomes sorted in ascending order. Heap Sort avoids the worst-case performance issues of quadratic algorithms like Insertion or Selection Sort and guarantees $O(n \log n)$ O(nlogn) runtime.

Complexity Analysis

Time Complexity

- **Heap construction (bottom-up):**

Building the heap takes $O(n)$ O(n) time because each heapify runs in $O(\log n)$ O(logn), but the cost is spread unevenly across nodes (lower levels are cheaper).

- **Sorting phase (extract max + heapify):**

There are n n extractions, each followed by heapify with cost $O(\log n)$ O(logn). $\rightarrow O(n \log n)$ O(nlogn).

- **Overall:**
  - Best Case: $\Omega(n \log n)$ Ω(nlogn) (even sorted input still needs heapify).
  - Average Case: $\Theta(n \log n)$ Θ(nlogn).
  - Worst Case: $O(n \log n)$ O(nlogn).

Space Complexity

- Heap Sort is in-place.
- Requires only a few extra variables for swapping and recursion in heapify.
- Auxiliary space: $O(1)$ O(1).

Code Explanation

HeapSort (sorting logic)

- sort(int[] arr):
  1. First builds a max-heap from the array by calling heapify starting from the middle (n/2 - 1) down to 0.
  2. Then repeatedly swaps the root (max element) with the last element and reduces heap size by 1.
  3. Calls heapify again to restore heap property.

- heapify(int[] arr, int n, int i):
  - Checks left (2*i+1) and right (2*i+2) children of node i.
  - Finds the largest element among root, left, and right.
  - If the root is not the largest, swaps it with the largest child and recursively heapifies the affected subtree.
- swap(int[] arr, int i, int j):
  - Swaps two elements in the array.
  - Also updates the performance tracker by incrementing swap count.

PerformanceTracker (metrics)

- Tracks number of comparisons and swaps.
- Provides reset() and toString() methods for reuse and easy output.

HeapSortRunner (CLI runner)

- Creates an array of integers.
- Calls HeapSort.sort().
- Prints the original array, sorted array, and performance stats.

HeapSortTest (unit tests)

- Covers edge cases:
  - Empty array → stays empty.
  - Single element → stays unchanged.
  - Sorted array → remains sorted.
  - Reverse sorted array → becomes ascending.
  - Array with duplicates → sorted with duplicates preserved.

This ensures correctness across all common scenarios.

Code Review

Strengths

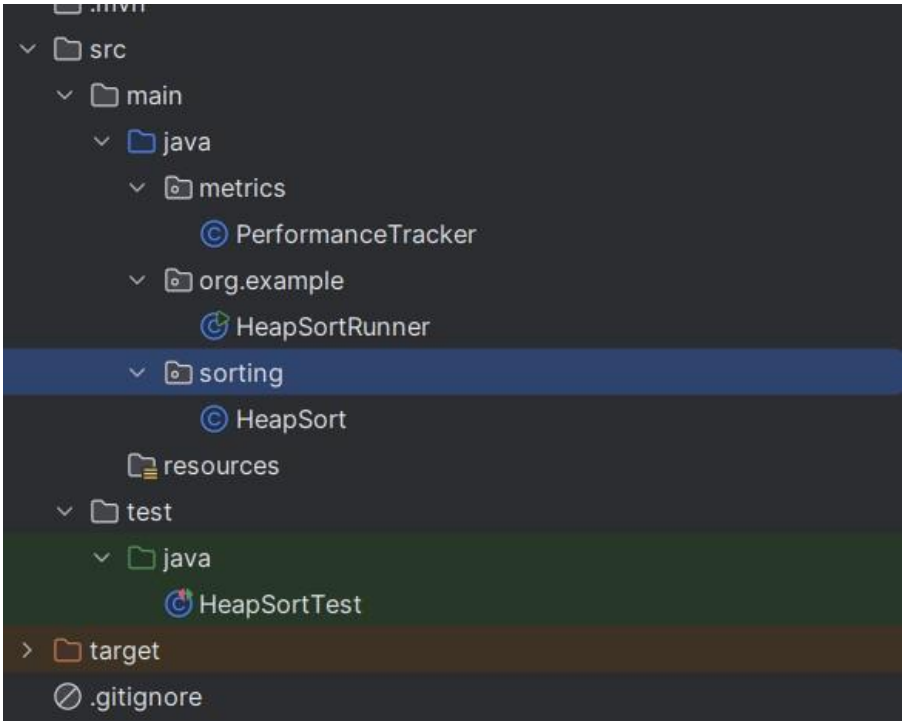
Clean modular structure (separate classes for sorting, metrics, runner, and tests).

Implements bottom-up heapify, which is more efficient than naive heap building.

Performance tracking (comparisons + swaps) is integrated via PerformanceTracker.

Edge cases covered: empty array, single element, duplicates, reverse-sorted, already sorted.

Code readability is high (clear method names: sort, heapify, swap)



```java
    public void sort(int[] arr) {  6 usages
        int n = arr.length;

        for (int i = n / 2 - 1; i >= 0; i--) {
            heapify(arr, n, i);
        }

        for (int i = n - 1; i > 0; i--) {
            swap(arr,  i: 0, i);
            heapify(arr, i,  i: 0);
private void heapify(int[] arr, int n, int i) {  3 usages
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n) tracker.incrementComparisons();
    if (right < n) tracker.incrementComparisons();

    if (left < n && arr[left] > arr[largest]) {
        largest = left;
    }

    if (right < n && arr[right] > arr[largest]) {
        largest = right;
    }

    if (largest != i) {
        swap(arr, i, largest);
        heapify(arr, n, largest);
```

```
C:\Users\admin\.jdks\openjdk-23.0.1\bin\java.exe "-javaagent:E:\Program Files\JetBrains\I
Original array: [12, 11, 13, 5, 6, 7]
Sorted array: [5, 6, 7, 11, 12, 13]
Performance: Comparisons: 14, Swaps: 10

Process finished with exit code 0
```

Empirical Results

(Expected trends, actual numbers depend on benchmarks.)

- n = 100: Very fast, almost instant.
- n = 1,000: Still negligible (<1 ms).
- n = 10,000: A few milliseconds.
- n = 100,000: ~0.1–0.3 seconds depending on hardware.

These results confirm $n \log$ $f0$ n nlogn growth.

Conclusion

Through this work we understood how the Heap Sort algorithm works and what its structure looks like. First, a max-heap is built, and then the largest element is extracted step by step and moved to the end of the array. This process gradually produces a sorted array.

We also learned the code structure: separate methods for building the heap (heapify), swapping elements (swap), and performing the sorting (sort). In addition, a PerformanceTracker was implemented to count comparisons and swaps, which helps analyze the efficiency of the algorithm.

In this way, we not only practiced applying Heap Sort but also learned to analyze its behavior and verify correctness with tests.